



CSE 560 | Spring 2025

Data Models and Query Languages**Homework 1****Topics covered:** *relational Model, Relational algebra, SQL***Name :** Taran Mamidala**UBIT ID :** 50604177**Problem 1**

- (a) Define the following terms: Relation, Relation Schema, Database Schema
- (b) A database administrator is tasked with implementing strict access control policies while ensuring efficient data management and recovery processes. Given the following scenarios, classify each operation as DDL, DML, or None and justify why it belongs to that category in detail.
- (i) A record is inserted into the "Employees" table, but the transaction is later rolled back due to an error.
 - (ii) A table named "Employees" is created with constraints ensuring that salary cannot be negative.

Answer :

- (a) A **relation** is a table within a database. It is made up of rows and columns where each row represents an individual record and each column represents an attribute of that record. It is known as a relation as a structured way to store data similar to how a spreadsheet organizes information.

A **relation schema** is the blueprint for a relation. It details the structure of the table by specifying the name of the relation, the names and data types of its columns, and any constraints or rules that the data in those columns must follow (such as a primary key constraint or a check constraint). The schema itself contains no data; it merely defines how data should be organized.

A **database schema** is the overall architecture of a database. It encompasses all of the relation schemas (or tables) and also defines the relationships between these tables (for example, through foreign keys). Additionally, a database schema might include definitions for views, indexes, and other database objects. In essence, the database schema provides a comprehensive map of how data is structured and interrelated across the entire database.

- (b) In the first scenario, a record is inserted into the "Employees" table but the transaction is later rolled back due to an error. This operation is classified as a Data Manipulation Language (DML) operation because it involves the manipulation of data within the existing table structure. Inserting a record is a typical DML action. The fact that the transaction is later rolled back due to an error does not change its classification; instead, it highlights the transactional nature of DML operations, where changes can be either committed or undone depending on the success of the transaction.

In the second scenario, a table named "Employees" is created with constraints ensuring that the salary cannot be negative. This operation falls under Data Definition Language (DDL) because it involves the creation of a new table along with its structural constraints. Creating a table is a schema modification action, and the inclusion of constraints to enforce data integrity is a key characteristic of DDL operations. DDL commands define the structure of the database and are not part of the regular data manipulation processes.

The first scenario is a DML operation because it involves data modification that is transactional in nature, while the second scenario is a DDL operation because it involves defining and altering the structure of the database.

Problem 2

Consider the database schema consists of five relations, whose schemas are: person(driverid, name, address)

car(license, plate, model, years) accident(report

number, year, location) owns(driverid, license plate)

participated(report number, license plate, driverid, damage, amount)

If required, insert synthetic data into each relation to answer the following Query :

1. Identify the primary key for each listed Relation.

Primary Keys:

- person: driverid
- car: plate
- accident: report_number
- owns: (driverid, license_plate)
- participated: (report_number, license_plate, driverid)

2. Write SQL Query to find the model number and total number of cars where the year of the Car is less than 15 years and the maker of the license plate belongs to New York or California.

```
SELECT model, COUNT(*) AS total_cars
FROM car
WHERE years < 15
      AND license IN ('NY', 'CA')
GROUP BY model;
```

Data Output			Messages	Notifications
				
	model	total_cars		
	character varying (50)	bigint		
1	Model C	1		
2	Model X	3		
3	Model Y	2		

3. Write SQL Query to find the names of all drivers who own a car that was involved in an accident.

```

SELECT DISTINCT p.name
FROM person p
JOIN owns o ON p.driverid = o.driverid
JOIN participated pt ON o.license_plate = pt.license_plate;

```

Data Output		Messages	Notifications
≡ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL			
	name		
	character varying (100)		
1	Sophia Wilson		
2	Alice Johnson		
3	Emma Stone		
4	Chris Evans		
5	Jim Brown		
6	John Doe		
7	Jane Smith		

4. Write SQL Query to find the number of accidents each driver has been involved in and the total damage amount they have paid for accidents that occurred in the last 3 years.

```

SELECT p.driverid, p.name,
       COUNT(DISTINCT a.report_number) AS accident_count,
       SUM(pt.amount) AS total_damage
FROM person p
JOIN participated pt ON p.driverid = pt.driverid
JOIN accident a ON pt.report_number = a.report_number
WHERE a.year >= 2021
GROUP BY p.driverid, p.name;

```

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

	driverid [PK] integer	name character varying (100)	accident_count bigint	total_damage numeric
1	1	John Doe	2	700.00
2	2	Jane Smith	2	550.00
3	3	Jim Brown	1	400.00
4	4	Alice Johnson	1	150.00
5	6	Chris Evans	1	100.00
6	11	Sophia Wilson	1	50.00

5. Write a SQL Query to find all drivers who do not own any cars.

```

SELECT p.name
FROM person p
WHERE NOT EXISTS (
    SELECT 1
    FROM owns o
    WHERE o.driverid = p.driverid
);

```

Data Output		Messages	Notifications
≡ 📄 ▼ 📋 ▼ 🗑️ 🔍 ⬇️ 📈 SQL			
	name character varying (100) 		
1	Bob Williams		
2	Olivia Brown		
3	Noah Davis		
4	Mason Taylor		
5	Isabella Anderson		
6	Ethan Thomas		
7	Ava Martinez		

Problem 3

Answer each of the following questions briefly. The questions are based on the following relational schema:

Employee(emp_id, emp_name, age: integer, salary: real) Assignment(emp_id: integer, project_id: integer, hours_worked: integer)

Project(project_id: integer, project_name: string, budget: real, manager_id: integer) If required, insert synthetic data into each relation to answer the following Query :

- Write an SQL statement to add a new employee, Bob Smith, with emp_id = 301, age = 35, and salary = 75,000.

```
INSERT INTO Employee (emp_id, emp_name, age, salary)
VALUES (301, 'Bob Smith', 35, 75000);
```

Data Output **Messages** Notifications

INSERT 0 1

Query returned successfully in 66 msec.

- Write an SQL statement to give every employee a 15 percent raise.

```
UPDATE Employee
SET salary = salary * 1.15;
```

Data Output **Messages** Notifications

UPDATE 4

Query returned successfully in 116 msec.

- Write an SQL statement to delete the “AI Research” project. Given the referential integrity constraints you chose for this schema, explain what happens when this statement is executed.

```
DELETE FROM Project
WHERE project_name = 'AI Research';
```

Data Output **Messages** Notifications

DELETE 1

Query returned successfully in 51 msec.

With the **ON DELETE CASCADE** referential integrity rule on the Assignment table, deleting the "AI Research" project (project_id 20) causes all assignments that reference project_id 20 to be automatically deleted. If this cascading rule were not in place, the deletion would either be blocked by the foreign key constraint or require manual deletion of the dependent assignment rows first.

Problem 4

Consider the bank database. Give an expression in the relational algebra for each of the following queries:

- **branch:** *branch_name, branch_city, assets*
- **customer:** *ID, customer_name, customer_street, customer_city*
- **loan:** *loan_number, branch_name, amount*
- **borrower:** *ID, loan_number*
- **account:** *account_number, branch_name, balance*
- **depositor:** *ID, account_number*

1. Find each loan number with a loan amount greater than \$10000.

Selecting all tuples from the **loan** relation where the *amount* exceeds 10,000:

$$R = \sigma_{amount > 10000}(\text{loan})$$

Projecting the *loan_number* attribute from R :

$$\pi_{\text{loan_number}}(R)$$

Expression:

$$\pi_{\text{loan_number}}(\sigma_{amount > 10000}(\text{loan}))$$

This expression filters the **loan** relation to include only those loans with an amount greater than \$10,000, then extracts the *loan_number*.

2. Find the names of customers who have taken a loan from a branch in 'New York'.

Selecting branches in New York:

$$R_1 = \sigma_{\text{branch_city} = \text{New York}}(\text{branch})$$

Join with the loan relation:

$$R_2 = \text{loan} \bowtie_{\text{loan.branch_name} = R_1.\text{branch_name}} R_1$$

Join with the borrower relation:

$$R_3 = \text{borrower} \bowtie_{\text{borrower.loan_number} = R_2.\text{loan_number}} R_2$$

Join with the customer relation:

$$R_4 = \text{customer} \bowtie_{\text{customer.ID} = R_3.\text{ID}} R_3$$

Projection:

$$\pi_{\text{customer_name}}(R_4)$$

$$\pi_{\text{customer_name}} \left(\text{customer} \bowtie_{\text{customer.ID} = \text{borrower.ID}} \left(\text{borrower} \bowtie_{\text{borrower.loan_number} = \text{loan.loan_number}} \left(\text{loan} \bowtie_{\text{loan.branch_name} = \text{branch.branch_name}} \sigma_{\text{branch_city} = \text{New York}}(\text{branch}) \right) \right) \right)$$

3. Find the ID of each depositor who has an account with a balance greater than \$6000 at the “Uptown” branch.

Select Accounts:

$$R_5 = \sigma_{balance > 6000 \wedge branch_name = Uptown}(account)$$

Join with Depositor:

$$R_6 = depositor \bowtie_{depositor.account_number = R_5.account_number} R_5$$

Projection:

$$\pi_{ID}(R_6)$$

Expression:

$$\pi_{ID} \left(depositor \bowtie_{depositor.account_number = account.account_number} \sigma_{balance > 6000 \wedge branch_name = Uptown}(account) \right)$$

This expression first selects accounts with a balance over \$6000 at the Uptown branch, then joins with the **depositor** relation to identify the corresponding depositors, and finally projects their IDs.

Problem 5

Consider a university database schema as follows:

Schema:

- **Student:** *sid, sname, major, level, age*
- **Class:** *cname, meetsAt, room, fid*
- **Enrolled:** *sid, cname*
- **Faculty:** *fid, fname, deptid*
- **Department:** *deptid, dname, budget, hod_fid*

Solve the below questions using relational algebra.

- (a) (i) Find students who are enrolled in every class offered by their own major.

Join **Faculty** and **Class** on the attribute *fid*:

$$C = Faculty \bowtie_{Faculty.fid = Class.fid} Class$$

C now contains each class along with the corresponding faculty's *deptid*.

For a given student *s* with major *s.major*, the set of classes offered by that department is:

$$R(s) = \pi_{cname}(\sigma_{deptid = s.major}(C))$$

For student s , the set of classes in which they are enrolled is:

$$S(s) = \pi_{\text{cname}}(\sigma_{\text{sid}=s.\text{sid}}(\text{Enrolled}))$$

Student s qualifies if $S(s)$ contains every class in $R(s)$, which is expressed as:

$$S(s) \div R(s) = R(s)$$

The set of students satisfying the condition is:

$$\begin{aligned} & \{ (s.\text{sid}, s.\text{sname}) \mid s \in \text{Student and } \pi_{\text{cname}}(\sigma_{\text{sid}=s.\text{sid}}(\text{Enrolled})) \div \pi_{\text{cname}}(\sigma_{\text{deptid}=s.\text{major}}(C)) \\ & = \pi_{\text{cname}}(\sigma_{\text{deptid}=s.\text{major}}(C)) \} \end{aligned}$$

In words, we select those students s for whom the set of classes they are enrolled in covers all classes offered by their own major's department.

(ii) Find faculty members who teach classes in all rooms where some class is conducted.

Let

$$R = \pi_{\text{room}}(\text{Class})$$

be the set of all rooms where classes are conducted.

For a given faculty member f , the set of rooms in which they teach is:

$$T(f) = \pi_{\text{room}}(\sigma_{\text{fid}=f.\text{fid}}(\text{Class}))$$

Faculty member f qualifies if $T(f)$ contains all rooms in R , i.e.,

$$T(f) \div R = R$$

The set of qualifying faculty members is:

$$\{ (f.\text{fid}, f.\text{fname}) \mid f \in \text{Faculty and } \pi_{\text{room}}(\sigma_{\text{fid}=f.\text{fid}}(\text{Class})) \div \pi_{\text{room}}(\text{Class}) = \pi_{\text{room}}(\text{Class}) \}$$

This picks out those faculty members who have taught in every room where any class is held.

(b) Find students who are not enrolled in any class taught by a faculty member from the student's department.

For each student s , consider the relation:

$$R = \pi_{\text{cname}} \left(\sigma_{\text{deptid}=s.\text{major}} \left(\text{Faculty} \bowtie_{\text{Faculty.fid}=\text{Class.fid}} \text{Class} \right) \right)$$

For student s , the classes they are enrolled in are:

$$S = \pi_{\text{cname}}(\sigma_{\text{sid}=s.\text{sid}}(\text{Enrolled}))$$

get those students for which the intersection $S \cap R$ is empty. Equivalently, express the answer as the set difference:

$$\pi_{\text{sid}, \text{sname}}(\text{Student}) \setminus \pi_{\text{sid}, \text{sname}}(\text{Student} \bowtie (R \bowtie \text{Enrolled}))$$

Thus, the desired set of students is:

$$\pi_{sid, sname}(\text{Student}) \setminus \pi_{sid, sname} \left(\text{Student} \bowtie \left(\sigma_{\text{deptid}=\text{major}} \left(\text{Faculty} \bowtie_{\text{Faculty.fid}=\text{Class.fid}} \text{Class} \right) \bowtie \text{Enrolled} \right) \right)$$

This expression returns those students who are *not* enrolled in any class taught by a faculty member from their own department.

- (c) Find students who are enrolled in at least two different classes that meet at the same time.

Join Enrolled and Class:

Form the relation:

$$E = \text{Enrolled} \bowtie_{\text{Enrolled.cname}=\text{Class.cname}} \text{Class}$$

Rename for Self-Join:

Rename E into two copies, E_1 and E_2 , to compare different enrollments for the same student.

Select Pairs Meeting at the Same Time:

Select pairs of tuples where the same student is enrolled in two different classes that meet at the same time:

$$R = \sigma_{E_1.sid=E_2.sid \wedge E_1.meetsAt=E_2.meetsAt \wedge E_1.cname \neq E_2.cname} (E_1 \times E_2)$$

$$\pi_{sid}(R)$$

$$\pi_{sid} \left(\sigma_{E_1.sid=E_2.sid \wedge E_1.meetsAt=E_2.meetsAt \wedge E_1.cname \neq E_2.cname} (\rho_{E_1}(E) \times \rho_{E_2}(E)) \right)$$

where $E = \text{Enrolled} \bowtie_{\text{Enrolled.cname}=\text{Class.cname}} \text{Class}$.

Problem 6

Considering the given data base schemas and sample entries to answer the SQL queries below.

Questions:

- List all patients who have appointments in the "Cardiology" department.

```
SELECT DISTINCT p.*
FROM Patients p
JOIN Appointments a ON p.patient_id = a.patient_id
JOIN Doctors d ON a.doctor_id = d.doctor_id
JOIN Departments dep ON d.department_id = dep.department_id
WHERE dep.department_name = 'Cardiology';
```

Data Output Messages Notifications

	patient_id [PK] integer	name character varying (100)	date_of_birth date	gender character (1)	phone character varying (15)
1	1	Alice Brown	1985-03-15	F	555-1234
2	3	Charlie White	1978-11-05	M	555-8765

- Count the number of appointments per month in 2023.

```

SELECT DATE_TRUNC('month', appointment_datetime) AS month,
       COUNT(*) AS appointment_count
FROM Appointments
WHERE EXTRACT(YEAR FROM appointment_datetime) = 2023
GROUP BY month
ORDER BY month;

```

Data Output Messages Notifications			
≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL			
	month	appointment_count	
	timestamp without time zone	bigint	
1	2023-10-01 00:00:00	4	

3. Find doctors who have not had any appointments in the last 30 days.

```

SELECT d.*
FROM Doctors d
WHERE NOT EXISTS (
    SELECT 1
    FROM Appointments a
    WHERE a.doctor_id = d.doctor_id
          AND a.appointment_datetime >= (CURRENT_DATE - INTERVAL '30 days')
);

```

Data Output Messages Notifications				
≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL				
	doctor_id	name	specialization	department_id
	[PK] integer	character varying (100)	character varying (50)	integer
1	202	Dr. Bob Johnson	Neurologist	102
2	203	Dr. Eva Lee	Cardiologist	101
3	201	Dr. Alice Smith	Cardiologist	101

4. Identify patients with more than one appointment on the same day.

```

SELECT patient_id,
       DATE(appointment_datetime) AS appt_date,
       COUNT(*) AS num_appointments
FROM Appointments
GROUP BY patient_id, DATE(appointment_datetime)
HAVING COUNT(*) > 1;

```

Data Output Messages Notifications			
≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL			
	patient_id	appt_date	num_appointments
	integer	date	bigint
1	3	2023-10-03	2

5. Find the most frequently prescribed medication.

```
SELECT medication, COUNT(*) AS frequency
FROM Prescriptions
GROUP BY medication
ORDER BY frequency DESC
LIMIT 1;
```

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 1

Page No: 1 of 1

	medication character varying (100)	frequency bigint
1	Amlodipine	1